

PRATICAL WEEK12

TASK1:

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an empty buffer slot */
        sem_wait(&empty);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        /* Increase number of full slots */
        sem_post(&full);
    }
}
```

```

        NULL);

/* Create consumer */
pthread_create(&consumer_thread,
               NULL,
               consumer,
               NULL);

/* Wait for both threads */
pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_t end = clock();

double time_taken =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("Buffer size: %d\n", buffer_size);
printf("Items processed: %d\n", ITEMS);
printf("Time taken: %f seconds\n", time_taken);

if (time_taken > 0)
{
    printf("Throughput: %.2f items/second\n",
           ITEMS / time_taken);
}

/* Destroy synchronization objects */
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

free(buffer);

return 0;
}

```

```
laasya@laasya: ~/producer_consumer
laasya@laasya:~$ mkdir producer_consumer
laasya@laasya:~$ cd producer_consumer
laasya@laasya:~/producer_consumer$ v
v: command not found
laasya@laasya:~/producer_consumer$ nano producer_consumer.c
laasya@laasya:~/producer_consumer$ gcc producer_consumer.c -o producer_consumer -pthread
laasya@laasya:~/producer_consumer$ ./producer_consumer 5
Consumer sum: 4999950000
Buffer size: 5
Items processed: 100000
Time taken: 0.869035 seconds
Throughput: 115070.16 items/second
laasya@laasya:~/producer_consumer$ ./producer_consumer 1
Consumer sum: 4999950000
Buffer size: 1
Items processed: 100000
Time taken: 2.358634 seconds
Throughput: 42397.42 items/second
laasya@laasya:~/producer_consumer$ ./producer_consumer 5
Consumer sum: 4999950000
Buffer size: 5
Items processed: 100000
Time taken: 0.672621 seconds
Throughput: 148672.13 items/second
laasya@laasya:~/producer_consumer$ ./producer_consumer 10
Consumer sum: 4999950000
Buffer size: 10
Items processed: 100000
Time taken: 0.374144 seconds
Throughput: 267276.77 items/second
laasya@laasya:~/producer_consumer$ ./producer_consumer 100
Consumer sum: 4999950000
Buffer size: 100
Items processed: 100000
Time taken: 0.081036 seconds
Throughput: 1234019.45 items/second
laasya@laasya:~/producer_consumer$ |
```

TASK2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1...\n");

    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locked Resource 1.\n");

    sleep(1);

    printf("Thread 1: Trying to lock Resource 2...\n");

    pthread_mutex_lock(&resource2);

    printf("Thread 1: Locked Resource 2.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: Trying to lock Resource 2...\n");

    pthread_mutex_lock(&resource2);

    printf("Thread 2: Locked Resource 2.\n");
```

```

sleep(1);

printf("Thread 2: Trying to lock Resource 1...\n");

pthread_mutex_lock(&resource1);

printf("Thread 2: Locked Resource 1.\n");

pthread_mutex_unlock(&resource1);
pthread_mutex_unlock(&resource2);

return NULL;
}

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL,
                  thread1_function, NULL);

    pthread_create(&thread2, NULL,
                  thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Locking Resource 1...\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Resource 1 locked.\n");
    sleep(1);
    printf("Thread 1: Locking Resource 2...\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Resource 2 locked.\n");
    printf("Thread 1: Using both resources.\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    printf("Thread 1: Resources released.\n");
    return NULL;
}

void *thread2_function(void *arg)
{
    /*
     * Resource ordering:

```

```

    printf("Thread 2: Resource 2 locked.\n");

    printf("Thread 2: Using both resources.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Resources released.\n");

    return NULL;
}

int main()
{
    pthread_t thread1;
    pthread_t thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL,
                  thread1_function, NULL);

    pthread_create(&thread2, NULL,
                  thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed without deadlock.\n");

    return 0;
}

```

```
laasya@laasya: ~/deadlock_la × + ▾  
laasya@laasya:~$ mkdir deadlock_lab  
laasya@laasya:~$ cd deadlock_lab  
laasya@laasya:~/deadlock_lab$ nano deadlock.c  
laasya@laasya:~/deadlock_lab$ gcc deadlock.c -o deadlock -pthread  
laasya@laasya:~/deadlock_lab$ ./deadlock  
Thread 2: Trying to lock Resource 2...  
Thread 2: Locked Resource 2.  
Thread 1: Trying to lock Resource 1...  
Thread 1: Locked Resource 1.  
Thread 2: Trying to lock Resource 1...  
Thread 1: Trying to lock Resource 2...  
^C  
laasya@laasya:~/deadlock_lab$ nano deadlock_prevention.c  
laasya@laasya:~/deadlock_lab$ gcc deadlock_prevention.c -o deadlock_prevention -pthread  
laasya@laasya:~/deadlock_lab$ ./deadlock_prevention  
Thread 1: Locking Resource 1...  
Thread 1: Resource 1 locked.  
Thread 2: Locking Resource 1...  
Thread 1: Locking Resource 2...  
Thread 1: Resource 2 locked.  
Thread 1: Using both resources.  
Thread 1: Resources released.  
Thread 2: Resource 1 locked.  
Thread 2: Locking Resource 2...  
Thread 2: Resource 2 locked.  
Thread 2: Using both resources.  
Thread 2: Resources released.  
Program completed without deadlock.  
laasya@laasya:~/deadlock_lab$ |
```